# Reviewer Pack — Permutation Polynomial Width Conjecture

Entry ce6a6c40a567 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 21:07:37 UTC

## Permutation Polynomial Width Conjecture

**Entry ID**: `ce6a6c40a567` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 21:07:37 UTC

### 1. Conjecture

**Field A** (mathematical branch): Permutation polynomials over finite fields **Field B** (complexity object): Branching program width

**Statement**:

> The minimal width of a branching program computing a permutation polynomial of degree d over GF(q) is Θ(log d).

**Rationale (proposer's reasoning)**:

> Permutation polynomials encode combinatorial symmetries that influence branching program width via their algebraic structure, linking finite field arithmetic to $NC^1$ complexity.

**Taxonomy category**: `BARRINGTON_ALG` (status at proposal time: partially_alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `7450e05feebfd57d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> default: >=80% seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.95 | SAFE | SAFE |
| NATURAL_PROOFS | SAFE | 0.95 | SAFE | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.00 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.95 | SAFE | SAFE |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

### 5.1 Generated Python source

```python
import random
import math

def generate_gf_elements(q):
    elements = [i for i in range(q)]
    return elements

def is_prime(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def generate_primes(min_val, max_val):
    primes = []
    for num in range(min_val, max_val + 1):
        if is_prime(num):
            primes.append(num)
    return primes

def generate_random_polynomial(degree, q):
    coefficients = [random.randint(0, q - 1) for _ in range(degree + 1)]
    return coefficients

def evaluate_polynomial(poly, x, q):
    result = 0
    power = 1
    for coeff in poly:
        result = (result + coeff * power) % q
        power = (power * x) % q
    return result

def is_permutation_poly(poly, q):
    elements = generate_gf_elements(q)
    image_set = set()
    for element in elements:
        value = evaluate_polynomial(poly, element, q)
```

```
        if value in image_set:
            return False
        image_set.add(value)
    return len(image_set) == q

def compute_branching_program_width(poly, q):
    n = len(poly) - 1
    dp = [[0] * (q + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        dp[i][0] = 1
        for j in range(1, q + 1):
            dp[i][j] = dp[i - 1][(j * poly[-2]) % q]
    return max(max(row) for row in dp)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    q = random.choice([2**k for k in range(3, 6)])  # GF(8), GF(16), GF(32)
    d = random.randint(5, 40)
    poly = generate_random_polynomial(d, q)

    if not is_permutation_poly(poly, q):
        return {
            "metric_name": "branching_program_width",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "not_a_permutation"
        }

    width = compute_branching_program_width(poly, q)
    log_d = math.log2(d) if d > 0 else 0

    return {
        "metric_name": "branching_program_width",
        "metric_value": width,
        "instances_tested": 1,
        "conjecture_holds": abs(width - log_d) < 1e-6,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or generate_primes(2, 100)[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_width = (sum((r["metric_value"] - mean_width)**2 for r in results if r["metric_value"] is
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_frac
```

```
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"not_a_permutation\" first_failing_seed={first_f
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
olds': False, 'counterexample': 'not_a_permutation'}
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': None, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': None, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': None, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': 1, 'instances_tested': 1, 'conject
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': None, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': None, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': None, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': None, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': 1, 'instances_tested': 1, 'conject
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': 1, 'instances_tested': 1, 'conject
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': None, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'branching_program_width', 'metric_value': None, 'instances_tested': 1, 'conj
RESULT: FALSIFIED counterexample="not_a_permutation" first_failing_seed=11
```

## 8. Critic adversarial review

**Critic verdict**: `CHALLENGE`

**Critic reasoning**:

> All trials have instances_tested=1, which is insufficient to establish scaling behavior. The 'not_a_permutation' counterexample suggests the test instances may not be valid permutation polynomials, violating the conjecture's premise. The metric_value=None entries indicate potential measurement errors or invalid computations.

## 9. Final verdict & safety rail

**Verdict**: `INCONCLUSIVE`

**Reasoning**:

> The test failed to meet the pre-registered support fraction threshold (80%) and the critic challenges the validity of the counterexample due to insufficient instance testing (all trials tested 1 instance). The 'not_a_permutation' counterexample may indicate invalid test cases rather than a true falsification. | next: Run tests with ≥100 instances across varying d and q to validate scaling behavior and confirm permutation polynomial validity

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | qwen3:8b | 0 | 0 | 109606 |
| 2 | propose | ollama_remote | qwen3:8b | 0 | 0 | 128541 |

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 3 | preregistration | ollama_remote | qwen3:8b | 0 | 0 | 24147 |
| 4 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 20770 |
| 5 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 13593 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 15222 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11364 |
| 8 | critic | ollama_remote | qwen3:8b | 0 | 0 | 25016 |
| 9 | judge | ollama_remote | qwen3:8b | 0 | 0 | 17790 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 366048 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/ce6a6c40a567.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ce6a6c40a567.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/ce6a6c40a567.tar.gz` (if generated)